

Reviewer Pack — Arithmetic Progression Invariance of Communication Complexit...

Entry 3bb096ac85fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 19:27:15 UTC

Arithmetic Progression Invariance of Communication Complexity under Shifts

Entry ID: 3bb096ac85fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 19:27:15 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (specifically, Arithmetic Progressions) **Field B** (complexity object): Communication Complexity

Statement:

For any two Boolean functions f and g with the same communication complexity, if there exists an integer k such that the arithmetic progression $\{f(x+k), g(x+k)\}$ has no common elements for all x in the domain, then the communication complexity of f is at most twice that of g .

Rationale (proposer's reasoning):

Arithmetic progressions provide a way to study the structure of functions and their interactions, which may reveal hidden complexities in communication processes. This conjecture suggests that certain patterns in arithmetic progressions can be used to bound communication complexity, potentially leading to new insights into the design of efficient protocols.

Taxonomy category: ArithmeticProgressionCommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed4d121450cb8889

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given pair of Boolean functions f and g with equal communication complexity, if an arithmetic progression $\{f(x+k), g(x+k)\}$ for $k = 0, \dots, n-1$ has no common elements, we will accept that the communication complexity of f is at most twice that of g .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "arithmetic progression" AND "communication complexity" - "shifts in number theory" AND communication complexity" - "invariant property" IN title:arXiv:"number theory" AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1706.02207v3] On The Communication Complexity of High-Dimensional Permutations - [http://arxiv.org/abs/2505.01587v1] More efficient sifting for grid norms, and applications to multiparty communication complexity - [http://arxiv.org/abs/2102.00421v2] Larger Corner-Free Sets from Better NOF Exactly- N Protocols - [http://arxiv.org/abs/2505.20998v2] Compression and complexity for sumset sizes in additive number theory - [http://arxiv.org/abs/2405.15368v1] Complexity of Robust Orbit Problems for Torus Actions and the abc-conjecture - [http://arxiv.org/abs/1710.08602v2] On the Polynomial Parity Argument Complexity of the Combinatorial Nullstellensatz - [s2:2505.01587] More efficient sifting for grid norms, and applications to multiparty communication complexity - [s2:10.1109/IHCSP63227.2024.10959753] Machine Mentality and its Progression Through the Complexity of Mathematical Modules

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.randint(0, 1) for _ in range(n)]

def communication_complexity(f):
    n = len(f)
    cc = 0
    for i in range(n):
        if f[i] == 1:
            cc += 1
    return cc

def arithmetic_progression(f, g, k, n):
    ap = [f[i + k] != g[i + k] for i in range(n)]
    return all(x for x in ap)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = 30
f = generate_boolean_function(n)
g = generate_boolean_function(n)

cc_f = communication_complexity(f)
cc_g = communication_complexity(g)

if cc_f != cc_g:
    return {
        "metric_name": "communication_complexity",
        "metric_value": 0.0,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": f"Communication complexities differ: {cc_f} != {cc_g}"
    }

ap_no_common_elements = all(arithmetic_progression(f, g, k, n) for k in range(n))

if ap_no_common_elements:
    return {
        "metric_name": "communication_complexity",
        "metric_value": min(cc_f, 2 * cc_g),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "communication_complexity",
        "metric_value": 0.0,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Arithmetic progression has common elements"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:

```

```

if not r["conjecture_holds"]:
    counterexample = r["counterexample"]
    first_failing_seed = seed
    break

print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

AL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_max':
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0.0, 'instances_tested': 1, 'n_ma
RESUL

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test uses $n = 30$, which is too small to be conclusive. The conjecture may hold for larger values of n , and the current results do not scale trivially with n .

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test provides a counterexample where communication complexities differ: $13 \neq 14$, indicating that the conjecture does not hold. | next: Increase the size of the domain (n) and retest the conjecture to determine if it holds for larger values.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22283
2	preregistration	ollama_remote	glm4:latest	0	0	9444
3	novelty	ollama_remote	glm4:latest	0	0	8404
4	novelty	ollama_remote	glm4:latest	0	0	29547
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10957
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18817
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11187

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8620
9	critic	ollama_remote	glm4:latest	0	0	17343
10	judge	ollama_remote	glm4:latest	0	0	21929

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 158532 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/3bb096ac85fc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3bb096ac85fc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3bb096ac85fc.tar.gz (if generated)